

좋습니다. 기존 `stock_analyzer.py` 에 **ADX, DI+, DI-**를 추가하면 됩니다.

핵심 개념은 아주 간단합니다. **DI+** 는 상승 방향의 힘, **DI-** 는 하락 방향의 힘을 보고, **ADX** 는 “방향과 상관없이 추세가 얼마나 강한가”를 봅니다.

보통 해석은 이렇게 합니다.

- **DI+ > DI-** → 상승 우위
- **DI- > DI+** → 하락 우위
- **ADX < 20** → 추세 약함
- **ADX 20~25 이상** → 추세 형성
- **ADX 30 이상** → 비교적 강한 추세

기존 파일에 아래 함수를 추가하세요.

Python

실행됨

```
# -----
# 7. ADX / DI+ / DI-
# -----

def calculate_adx(
    df: pd.DataFrame,
    period: int = 14
) -> pd.DataFrame:

    df = df.copy()

    # 이전 고가/저가와와의 차이
    high_diff = df["High"].diff()
    low_diff = -df["Low"].diff()

    # +DM
    plus_dm = np.where(
        (high_diff > low_diff) & (high_diff > 0),
        high_diff,
        0.0
    )

    # -DM
    minus_dm = np.where(
        (low_diff > high_diff) & (low_diff > 0),
        low_diff,
        0.0
    )

    plus_dm = pd.Series(
        plus_dm,
        index=df.index
```



```

        / atr
    )

    df["DI_MINUS"] = (
        100
        * minus_dm_smoothed
        / atr
    )

# DX 계산
dx = (
    (
        df["DI_PLUS"]
        - df["DI_MINUS"]
    ) .abs()
    /
    (
        df["DI_PLUS"]
        + df["DI_MINUS"]
    )
) * 100

# ADX
df["ADX"] = dx.ewm(
    alpha=1 / period,
    adjust=False
).mean()

```

```
return df
```

그리고 기존 `calculate_indicators()` 에 한 줄 추가합니다.

Python

실행됨

```

def calculate_indicators(
    df: pd.DataFrame
) -> pd.DataFrame:

    df = calculate_moving_averages(df)

    df = calculate_rsi(df)

    df = calculate_macd(df)

    df = calculate_atr(df)

    df = calculate_volume(df)

    df = calculate_adx(df)

    return df

```

이제 `analyze_technical()` 도 ADX와 DI를 사용하도록 확장해봅시다.

Python

실행됨

```
def analyze_technical(
    df: pd.DataFrame
) -> dict:

    latest = df.iloc[-1]

    result = {}

    # -----
    # 이동평균
    # -----

    if (
        latest["Close"] > latest["MA20"]
        and latest["MA20"] > latest["MA60"]
    ):
        result["trend"] = "bullish"

    elif (
        latest["Close"] < latest["MA20"]
        and latest["MA20"] < latest["MA60"]
    ):
        result["trend"] = "bearish"

    else:
        result["trend"] = "neutral"

    # -----
    # RSI
    # -----

    if latest["RSI"] >= 70:
        result["rsi_signal"] = "overbought"

    elif latest["RSI"] <= 30:
        result["rsi_signal"] = "oversold"

    else:
        result["rsi_signal"] = "neutral"

    # -----
    # MACD
    # -----

    if latest["MACD"] > latest["MACD_SIGNAL"]:
```

```

        result["macd_signal"] = "bullish"

    else:
        result["macd_signal"] = "bearish"

    # -----
    # Volume
    # -----

    if latest["VOLUME_RATIO"] >= 1.5:
        result["volume_signal"] = "high"

    else:
        result["volume_signal"] = "normal"

    # -----
    # ADX 추세 강도
    # -----

    if latest["ADX"] >= 30:
        result["trend_strength"] = "strong"

    elif latest["ADX"] >= 20:
        result["trend_strength"] = "moderate"

    else:
        result["trend_strength"] = "weak"

    # -----
    # DI 방향
    # -----

    if latest["DI_PLUS"] > latest["DI_MINUS"]:
        result["di_signal"] = "bullish"

    elif latest["DI_PLUS"] < latest["DI_MINUS"]:
        result["di_signal"] = "bearish"

    else:
        result["di_signal"] = "neutral"

    return result

```

이제 점수에도 넣어보겠습니다.

기존 `calculate_technical_score()` 를 이렇게 확장하세요.

```
def calculate_technical_score(
    df: pd.DataFrame
) -> int:

    latest = df.iloc[-1]

    score = 50

    # -----
    # Trend
    # -----

    if (
        latest["Close"] > latest["MA20"]
        and latest["MA20"] > latest["MA60"]
    ):
        score += 15

    elif (
        latest["Close"] < latest["MA20"]
        and latest["MA20"] < latest["MA60"]
    ):
        score -= 15

    # -----
    # RSI
    # -----

    if 50 <= latest["RSI"] <= 70:
        score += 10

    elif latest["RSI"] >= 75:
        score -= 10

    elif latest["RSI"] <= 30:
        score += 5

    # -----
    # MACD
    # -----

    if latest["MACD"] > latest["MACD_SIGNAL"]:
        score += 10

    else:
        score -= 10
```

```

# -----
# Volume
# -----

if latest["VOLUME_RATIO"] >= 1.5:
    score += 5

# -----
# DI 방향
# -----

if latest["DI_PLUS"] > latest["DI_MINUS"]:
    score += 10

else:
    score -= 10

# -----
# ADX 추세 강도
# -----

if latest["ADX"] >= 30:

    if latest["DI_PLUS"] > latest["DI_MINUS"]:
        score += 10

    else:
        score -= 10

elif latest["ADX"] >= 20:

    if latest["DI_PLUS"] > latest["DI_MINUS"]:
        score += 5

    else:
        score -= 5

score = max(
    0,
    min(100, score)
)

return score

```

여기서 중요한 논리가 하나 있습니다.

ADX 는 그 자체로 bullish/bearish가 아닙니다.

예를 들어:

```
ADX = 40
```

이면 강한 추세라는 뜻이지 상승추세라는 뜻은 아닙니다.

그래서 반드시 같이 봐야 합니다.

```
ADX = 40
DI+ > DI-
→ 강한 상승추세
```

반대로:

```
ADX = 40
DI- > DI+
→ 강한 하락추세
```

그래서 점수에서도 ADX 단독으로 +10 을 주지 않고 DI 방향과 결합했습니다.

마지막으로 `analyze_stock()` 의 출력에 ADX, DI+, DI-를 추가합니다.

Python

실행됨

```
def analyze_stock(
    ticker: str
) -> dict:

    df = get_stock_data(
        ticker
    )

    df = calculate_indicators(
        df
    )

    technical = analyze_technical(
        df
    )

    score = calculate_technical_score(
        df
    )

    signal = make_trading_signal(
        score
    )

    latest = df.iloc[-1]
```



```
        "volume_ratio": float(
            latest["VOLUME_RATIO"]
        )
    }
}

return result
```

결과는 대략 이런 식이 됩니다.

Python

실행됨

```
{
  "ticker": "005930.KS",

  "price": 85000.0,

  "technical_score": 85,

  "signal": "BUY",

  "analysis": {

    "trend": "bullish",

    "rsi_signal": "neutral",

    "macd_signal": "bullish",

    "volume_signal": "normal",

    "trend_strength": "strong",

    "di_signal": "bullish"
  },

  "indicators": {

    "MA5": 84500.0,

    "MA20": 82000.0,

    "MA60": 79000.0,

    "RSI": 62.3,

    "MACD": 950.0,

    "ATR": 2100.0,

    "ADX": 34.2,
```

```

        "DI_PLUS": 29.5,

        "DI_MINUS": 16.8,

        "volume_ratio": 1.31
    }
}

```

이걸 자연어로 읽으면:

현재 주가는 MA20과 MA60 위

→ 상승추세

MACD > Signal

→ 상승 모멘텀

DI+ > DI-

→ 상승 방향 우세

ADX = 34

→ 추세 강함

따라서:

강한 상승추세

가 됩니다.

여기까지 오면 첫 번째 Python 기술적 분석기는 꽤 괜찮은 뼈대가 됩니다.

```

Price
|
├─ MA5 / MA20 / MA60
├─ RSI
├─ MACD
├─ ATR
├─ Volume
├─ DI+
├─ DI-
└─ ADX
    ↓
Technical Analysis
    ↓
Technical Score
    ↓
BUY / HOLD / SELL

```

그리고 다음 단계로 넘어가기 전에 하나를 더 추가하는 것을 추천합니다. **ATR**은 계산만 하고 현재 점수에는 거의 사용하지 않고 있기 때문입니다. 다음에는 ATR을 이용해 **Stop Loss**, **Take**

`Profit`, `position risk`를 계산하는 아주 작은 **Risk** 함수를 만들어보면 좋습니다.

그렇게 하면 Python 함수 기반 단계에서 이미 **Analysis** → **Signal** → **Risk**의 골격이 만들어지고, 그 다음 2단계인 **LLM Tool Calling**으로 넘어가기가 훨씬 자연스럽습니다.